Bangla Sign Language Gesture Recognition: Static Image Classification and Bounded Sentence Recognition

Nazmul Hassan
North South University, Dhaka, Bangladesh
ORCID: 0000-0001-6116-017X
iam.najib@yahoo.com

Preprint

Abstract

This study evaluates static Bangla hand-sign classification and exploratory recognition of a bounded sentence vocabulary, extending an undergraduate thesis system. The surviving Python architecture is retrained once, while a frozen MobileNetV2 classifier provides a modest comparison. A separate experiment uses the openly licensed KU-BdSL USLD collection; its grouped label space is kept distinct from the historical dataset. Evaluation uses fixed seeded partitions, content audits, class metrics, confusion matrices and prediction-level evidence. The historical CNN obtains 3.38% on 148 held-out images; the frozen comparison obtains 52.70% on the same split. The separate public-data model obtains 99.58% on 240 held-out images. These image-level tests do not establish unseen-signer performance. The software accepts images and recorded files as well as cameras, and a fixture-video test verifies the pipeline without camera hardware. A separate exploratory ten-sentence classifier recognises 6 of ten supported test clips and rejects 8 of ten background clips. The results support constrained recognition experiments; continuous translation and performance with new signers require further evaluation.

1. Introduction

Accessible communication tools require reliable evidence as well as working interfaces. A recogniser for a small set of hand configurations may support a constrained input experiment, but it does not capture the movement, context and grammar of a natural sign language. We evaluate static class prediction and a separate, bounded sentence-recognition task.

The original system was described in a ScienceOpen preprint [1]. Differences between its methods narrative and the surviving implementation motivate an explicit reconstruction of the executable CNN. This study contributes audited data partitions, reproducible model comparisons, class-level error analysis and an exploratory temporal model. The original preprint is retained unchanged; the additional experiments are reported separately.

2. Related work and scope

The KU-BdSL authors provide an openly available collection designed for Bengali consonant recognition [2]. The accompanying versioned dataset offers an explicit licence and documented collection conditions [3]. It is useful here because its class layout can be downloaded, hashed and audited without treating an undocumented image bundle as an independent benchmark.

MobileNetV2 introduces inverted residuals and linear bottlenecks for efficient vision models [4]. Here it is used only as an established, frozen ImageNet feature extractor. A small new classifier is learned on top. Its frozen-backbone configuration provides a controlled, computationally economical transfer-learning baseline.

The historical and public collections have different image distributions and label spaces. We therefore compare architectures within the historical split and report the public-data experiment separately; cross-dataset score differences cannot isolate architectural improvement.

3. Data and provenance

The historical repository contains 1,005 labelled images in each of three representations: Gray, Resized and Otsu_again. They are not independent collections. The re-run uses Gray only; the other copies are excluded from training and evaluation. Its 36 numeric folder labels inherit a provisional display mapping from the original interface. The duplicate vowel for class 6 is assigned a provisional display correction; expert verification remains outstanding. Quantitative evaluation retains the original numeric identifiers.

The historical paper describes constrained camera capture and hand cropping, but the repository supplies no verified signer identifiers, capture-session identifiers or participant-consent documentation. This revision therefore does not assert a verified count of individual historical signers. It also does not infer the collection of depth data merely from a prose claim when the available image pipeline does not expose such measurements.

The new experiment downloads KU-BdSL version 4, USLD only. The publisher describes smartphone images from 39 participants and 30 grouped consonant classes. Several folder labels represent multiple written characters with a shared sign class. Their decimal Unicode identifiers are preserved, including grouped identifiers; they are not silently expanded into separate targets. The measured inventory is recorded alongside the download manifest.

4. Split protocol and dataset audit

Before splitting, the code decodes every candidate image, applies the exact evaluation preprocessing, and hashes the resulting float32 tensor with SHA256. Identical inputs are grouped together. A group with conflicting labels is excluded and logged. A group with one label contributes one representative, with duplicate paths retained in the manifest. Unreadable inputs are excluded explicitly. This removes exact input duplicates, but it cannot guarantee independence between visually similar captures.

Each class is shuffled with seed 2026. Test and validation each receive the nearest integer to 15% of that class, with at least one example; the remainder is used for training. All records retain paths, class IDs, content hashes and partition membership. The historical CNN and its transfer baseline use the same saved manifest; the trainer validates the current content against it before reuse.

The historical audit finds 1005 unique usable images, split into 709 training, 148 validation and 148 test images. The public-data audit finds 1500 unique usable images, split into 1020 training, 240 validation and 240 test images. Exact duplicate and conflict details are in the machine-readable inventories. These counts describe decoded, usable inputs, not estimates inferred from a nominal dataset size.

No image in the test manifest is used for fitting or validation. Augmentation is applied only to

historical training batches. The fixed 12-epoch schedule is specified in advance; there is no test-based checkpoint selection or search over random seeds. Examples in the showcase are selected after evaluation for illustration and never used to estimate accuracy.

The partition is image-level. Although the public dataset documents multiple participants, the downloaded folder labels do not provide signer IDs suitable for a verified signer-disjoint split. Shared signers, backgrounds or sessions may therefore cross partitions. Near duplicates may also survive the exact-hash check. These are material limits on the interpretation of the scores, not details remedied by setting a seed.

5. Shared preprocessing

All inputs are resized to 128×128 pixels using nearest-neighbour interpolation. Colour frames are converted from OpenCV BGR to grayscale, and Otsu thresholding produces a binary image. Pixel values are divided by 255 and a single channel is retained. The same function is used by training, evaluation, the CLI and the desktop interface. There is no separate preprocessing path that could silently change the input scale at deployment.

The surviving training script specifies 128-pixel inputs despite the historical paper's discussion of 64-pixel images. This revision follows the code for the CNN re-run and states the difference. The static classifier requires a suitable hand crop; automatic hand localisation is outside this image-classification pipeline.

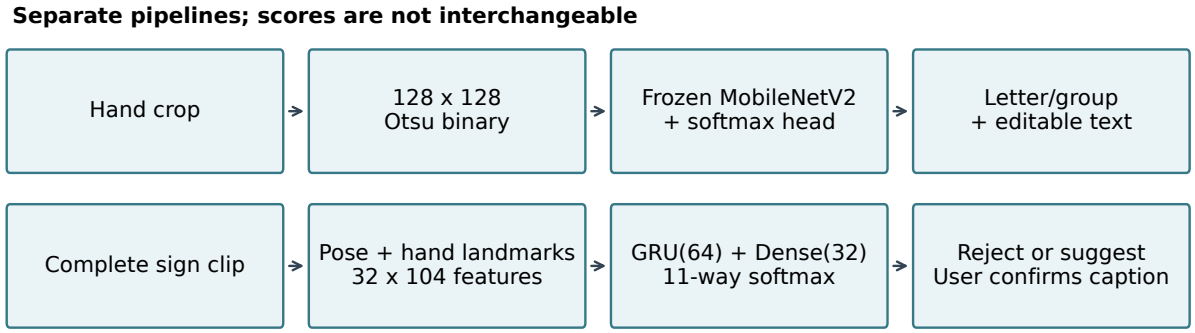


Figure 1: Inference architecture. Static classification and complete-clip sentence recognition use separate feature contracts and label spaces. The original CNN comparison is specified in the method text. Manual confirmation is an interface safeguard, not evidence of recognition accuracy.

6. Models and training setup

The reconstructed CNN uses two valid 5×5 convolutions with 32 filters, a 5×5 max-pooling layer and dropout 0.25. Two valid 3×3 convolutions with 64 filters follow, then 2×2 max pooling and dropout 0.25. Flattened features feed a 512-unit ReLU dense layer, dropout 0.5 and a softmax classifier. All convolutional layers use ReLU. The historical experiment has 36 outputs and preserves the architecture recorded in `ishara.py` at the thesis tag.

Adam uses its default learning rate of 0.001. Sparse categorical cross-entropy is minimised for 12 full epochs with batches of 18. Historical augmentation retains the surviving script's shear 0.2, zoom 0.2 and horizontal flips. These settings are preserved for traceability rather than endorsed as linguistically valid: a reflected sign can be inappropriate, and augmentation choices require review by sign-language users. Unlike the old fixed generator step counts, the modern loop visits

each training batch in an epoch.

The comparison repeats the binary input across three channels and scales it to $[-1, 1]$ for ImageNet MobileNetV2. Its backbone is frozen. Global average pooling produces 1,280-dimensional features, followed by dropout 0.2 and a new softmax head. Frozen features are cached once on CPU. The head is trained for 12 epochs using Adam, batch size 18 and no geometric augmentation. The exported end-to-end model still accepts image tensors, not cached vectors.

This is a small transfer baseline, not an architecture-only ablation: pretraining and augmentation differ. Both historical models nevertheless use precisely the same training, validation and test examples, making the difference in their measured outcomes inspectable. The public-data run trains another head in its distinct grouped class space.

Python, TensorFlow, Keras, OpenCV and other direct dependencies are pinned. TensorFlow random seeds and deterministic operations are enabled, and CPU execution is used. Environment metadata and model hashes accompany every experiment. A fixed seed improves repeatability within an environment; it does not guarantee bit-identical floating-point results across all processors and library versions.

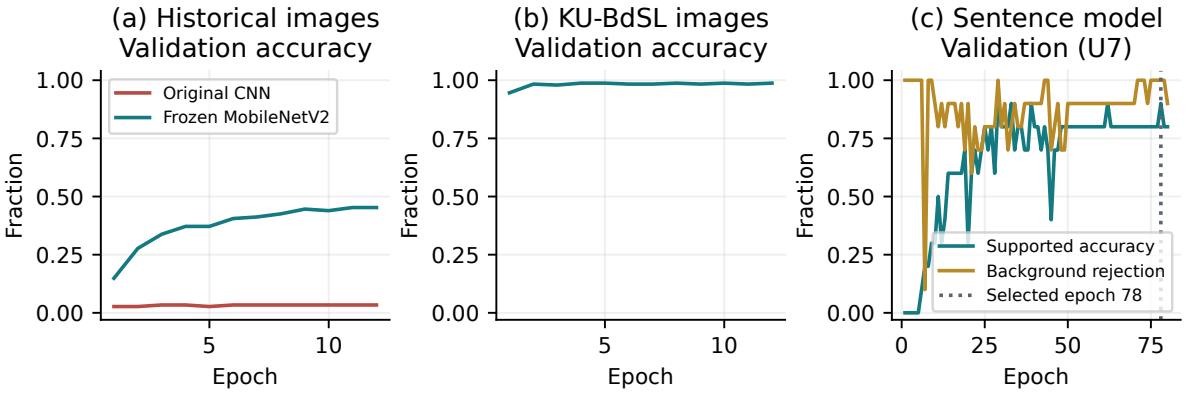


Figure 2: Unsmoothed validation histories from the saved runs. Panels (a) and (b) show all 12 image-training epochs. Panel (c) shows all 80 temporal epochs and the checkpoint selected using U7 validation. Test clips are not plotted or used for checkpoint selection. Training accuracy is omitted because the historical logger records the last batch rather than an epoch aggregate.

7. Metrics

Test predictions are obtained once by argmax of each output distribution. Accuracy is the number correct divided by test support. Per-class precision, recall and F1 are reported with support; undefined values are set to zero. The confusion matrices contain raw counts and use the saved class order. Softmax outputs are presented as uncalibrated scores, never as a validated probability of correctness.

8. Static classification results

The historical comparison measures two models on the same image-level split. The public-data row uses a different collection and grouped label space and should be interpreted separately. These are additional experimental measurements, reported independently of the original

Table 1: Static classification on fixed image-level test partitions. Historical models share one split; KU-BdSL is a separate task.

Experiment	Classes	Train	Val.	Test	Acc. (%)	Macro F1
Historical CNN	36	709	148	148	3.38	0.0018
Historical MobileNetV2	36	709	148	148	52.70	0.5072
KU-BdSL MobileNetV2	30	1020	240	240	99.58	0.9958

preprint's accuracy statements. One seed and one run per configuration do not establish robustness or a confidence interval over training variability.

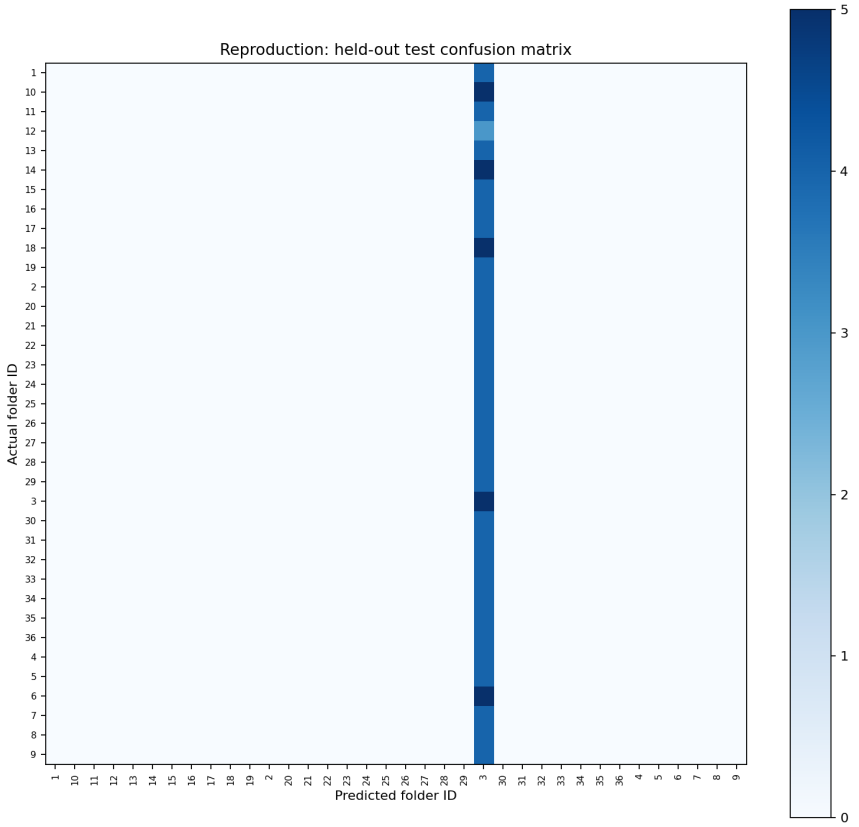


Figure 3: Historical CNN confusion matrix for 148 test images. Rows denote actual folder IDs and columns denote predicted IDs; entries are raw counts.

Figure 3 shows held-out predictions; the raw counts are available as CSV. Concentration in a small number of prediction columns indicates poor discrimination across the historical classes.

The public-data and historical tasks differ in image quality, class definitions, collection conditions and support per class. Cross-dataset score differences cannot isolate the effects of better software, representation learning or sample size. No comparison with a literature accuracy claim is used as evidence of improvement.

9. Per-class metrics

The following are the complete per-class precision (P), recall (R), F1 and support (N), rounded only for display. Unrounded values are in the committed CSV and JSON reports. The numeric IDs preserve the original folder labels; hyphenated public IDs denote grouped characters. Class order in the model and plots is lexicographic and stored explicitly.

Historical CNN

ID	P	R	F1	N
1	0.000	0.000	0.000	4
10	0.000	0.000	0.000	5
11	0.000	0.000	0.000	4
12	0.000	0.000	0.000	3
13	0.000	0.000	0.000	4
14	0.000	0.000	0.000	5
15	0.000	0.000	0.000	4
16	0.000	0.000	0.000	4
17	0.000	0.000	0.000	4
18	0.000	0.000	0.000	5
19	0.000	0.000	0.000	4
2	0.000	0.000	0.000	4
20	0.000	0.000	0.000	4
21	0.000	0.000	0.000	4
22	0.000	0.000	0.000	4
23	0.000	0.000	0.000	4
24	0.000	0.000	0.000	4
25	0.000	0.000	0.000	4
26	0.000	0.000	0.000	4
27	0.000	0.000	0.000	4
28	0.000	0.000	0.000	4
29	0.000	0.000	0.000	4
3	0.034	1.000	0.065	5
30	0.000	0.000	0.000	4
31	0.000	0.000	0.000	4
32	0.000	0.000	0.000	4
33	0.000	0.000	0.000	4
34	0.000	0.000	0.000	4
35	0.000	0.000	0.000	4
36	0.000	0.000	0.000	4
4	0.000	0.000	0.000	4
5	0.000	0.000	0.000	4
6	0.000	0.000	0.000	5
7	0.000	0.000	0.000	4
8	0.000	0.000	0.000	4
9	0.000	0.000	0.000	4

Public-data MobileNetV2

ID	P	R	F1	N
2433	1.000	1.000	1.000	8
2434	1.000	1.000	1.000	8
2435	1.000	1.000	1.000	8
2453	1.000	1.000	1.000	8
2454	1.000	1.000	1.000	8
2455	1.000	1.000	1.000	8
2456	1.000	1.000	1.000	8
2457	1.000	1.000	1.000	8
2458	1.000	1.000	1.000	8
2459	1.000	1.000	1.000	8
2460-2479	1.000	1.000	1.000	8
2461	1.000	1.000	1.000	8
2462	1.000	0.875	0.933	8
2463	1.000	1.000	1.000	8
2464	1.000	1.000	1.000	8
2465	1.000	1.000	1.000	8
2466	1.000	1.000	1.000	8
2467-2472	1.000	1.000	1.000	8
2468-2510	1.000	1.000	1.000	8
2469	1.000	1.000	1.000	8
2470	1.000	1.000	1.000	8
2471	1.000	1.000	1.000	8
2474	1.000	1.000	1.000	8
2475	1.000	1.000	1.000	8
2476-2477	1.000	1.000	1.000	8
2478	1.000	1.000	1.000	8
2480-2524-2525	1.000	1.000	1.000	8
2482	1.000	1.000	1.000	8
2486-2488-2487	0.889	1.000	0.941	8
2489	1.000	1.000	1.000	8

The transfer comparison's complete 36-class table is distributed as `reports/mobilenet/per_class.csv`. The public matrix appears below; its axis labels are numeric Unicode/group IDs rather than inferred letter names.

10. Error analysis and qualitative evidence

The examples above are the first correct and first incorrect public test predictions in the saved manifest, when each exists. Their source paths and full scores are stored in `examples.json`. Both originate from held-out images; they are illustrations rather than a random sample or an estimate of error frequency. Dataset credit: Jim, Rafi, Akon, Biswas and Nahid, KU-BdSL v4, CC BY 4.0. Images are resized derivatives.

Errors can be inspected at class level through recall and at prediction level through the complete results page. Low recall means many members of a class were missed; low precision means

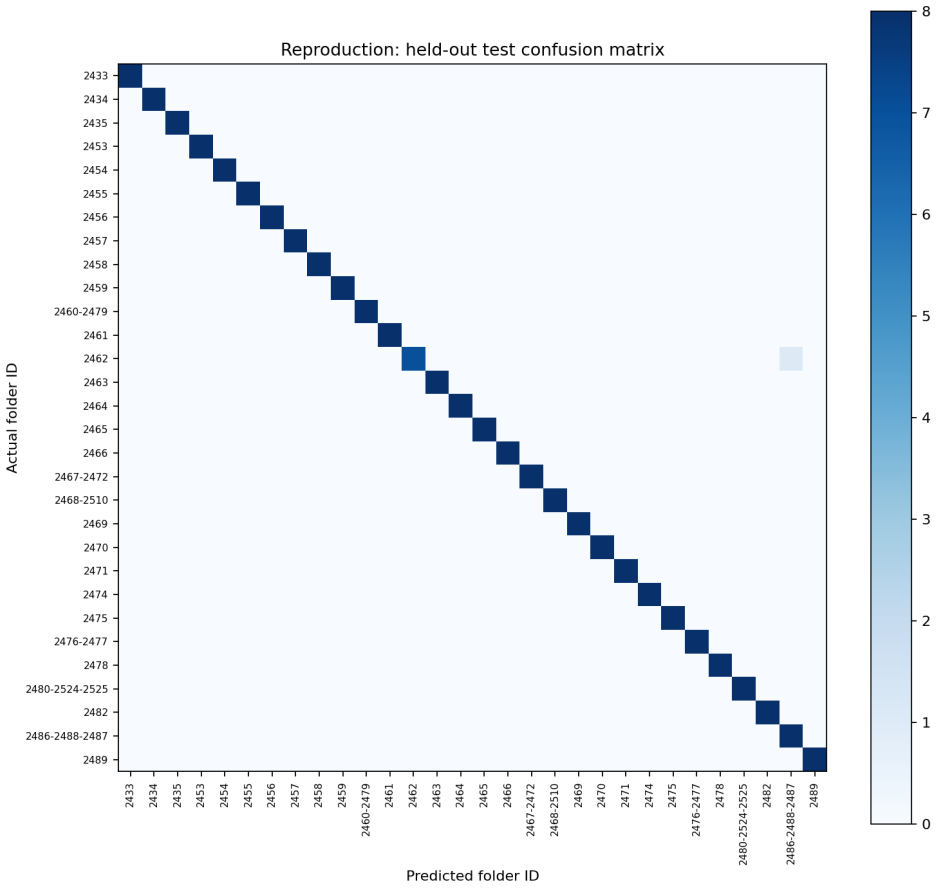


Figure 4: KU-BdSL MobileNetV2 confusion matrix for 240 test images in 30 grouped classes. Rows are actual IDs and columns predicted IDs.



Actual ID 2433; predicted 2433. Score 100.0%.



Actual ID 2462; predicted 2486-2488-2487. Score 25.5%.

Figure 5: A correct and an incorrect held-out KU-BdSL prediction. Scores are uncalibrated softmax outputs. Images: Jim et al., KU-BdSL v4, CC BY 4.0; resized derivatives.

predictions assigned to that class frequently belonged elsewhere. The released audit shows every test image and score, including failures, so readers can investigate competing hand configurations and background effects without relying on a few chosen successful screenshots.

The evidence does not establish a causal explanation for any individual error. Similar handshapes, threshold sensitivity, cropping and limited samples are plausible hypotheses for further investigation, not experimentally isolated causes. The binary preprocessing discards colour and texture; this may remove useful information and may interact poorly with features pretrained on natural colour images. An RGB baseline would be a useful future experiment, but it was not measured in this revision.

11. Implementation and software validation

The Python implementation provides command-line and PyQt5 interfaces for images, recorded video and cameras. Automated tests cover Otsu preprocessing, tensor contracts, image decoding, Keras prediction and video inference. Synthetic image and video fixtures test software behaviour independently of sign-recognition accuracy. Recorded-file sampling follows media timestamps, and tracking state resets between inputs.

The desktop video-input flow was also tested with a supported clip, a background false accept and a cropped BRAC tutorial greeting. The supported clip was recognised correctly; both unsupported inputs produced incorrect supported captions, including a coffee-related sentence for the greeting. Suggestions required confirmation before transcript insertion. No weights or thresholds were adjusted after these checks. The first two clips were previously evaluated examples, so this test measures integration consistency rather than new recognition accuracy. A separate letter-composition demonstration uses selected correctly classified held-out images; it is not continuous signing.

12. Moving-phrase feasibility experiment

The supplied Grameenphone and BRAC tutorials demonstrate moving words and sentences, which differ from isolated alphabet signs. A separate CPU prototype extracts upper-body and hand landmarks from signer crops, normalises coordinates using the shoulders, and compares resampled sequences with dynamic time warping. Printed caption pixels are not model features; they are used only to annotate reference and test intervals manually. Ten phrases have one reference example each from the first tutorial. The second tutorial contains a different signer and supplies four supported and eight unsupported test segments.

With a distance threshold of 0.20 and a relative margin of 0.10 specified before evaluation, the prototype recognises 1 of four supported phrases and rejects 4 of eight unsupported segments. These are tiny, manually segmented feasibility measurements, not continuous-transcription accuracy or a public-use validation. Its streaming preview and unknown state are software functions; they do not remedy the poor observed cross-signer result. More labelled examples, separate validation signers, representative unknown gestures and sign-language expert review are required before deployment.

Segment annotations and temporal errors are reported separately from the static image metrics.

12.1. Bounded sentence model

A follow-up experiment uses 100 selected clips from the public Ban-Sign-Sent-9K-V1 repository (`https://huggingface.co/datasets/banglagov/Ban-Sign-Sent-9K-V1`). Ten text-diverse target sentences and ten background sentence types were selected before recognition outcomes. Encoded signer IDs U4/U5/U6 supply 60 training clips; U7 supplies 20 validation clips; U19 supplies 20 test clips. There is one clip per sentence and ID. Identity and consent were not independently verified, and the inspected dataset card has no explicit licence; source footage is not redistributed.

An 11-class GRU classifier (64 units, dropout 0.3, dense 32, softmax including background) uses 32-step motion/shape landmark sequences, Adam 0.001, batch 8, 80 epochs, temporal warp/noise augmentation and seed 2026. Validation selects the checkpoint and rejection threshold, with DTW as an alternative backend. The selected GRU obtains 6/10 supported captions correct and rejects 8/10 background clips on U19; 6 of 10 accepted captions are correct. The

two false accepts illustrate the danger of confident captions. Background sentence types occur in training, so rejection does not establish novel-utterance detection. An earlier conservative baseline rejected all clips; its U19 outcome was already observed before this follow-up, which is exploratory rather than a wholly blind confirmation. Full clip-level predictions, class metrics and confusion counts accompany the code. Automatic sentence boundaries and general conversation translation remain unevaluated.

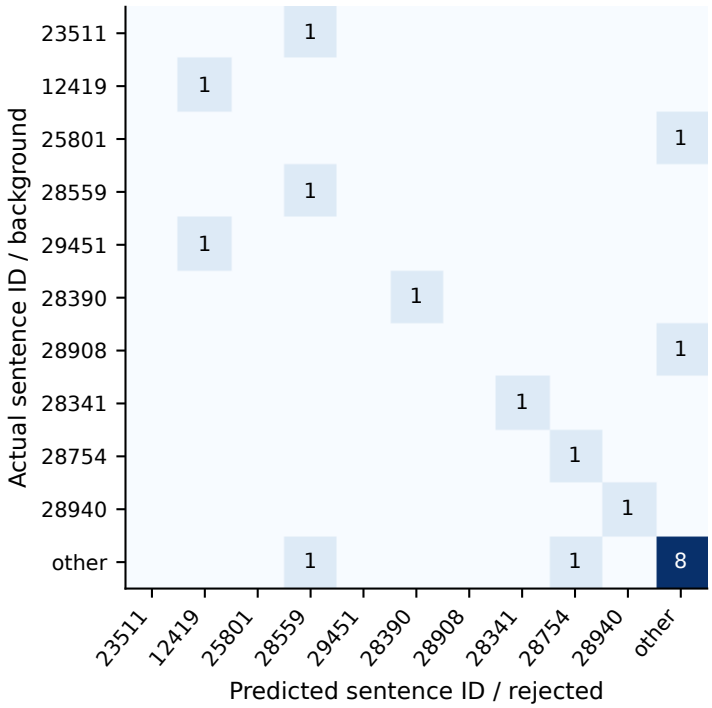


Figure 6: Raw confusion counts for 20 U19 sentence-test clips: one per supported sentence and ten background clips. Numeric IDs correspond to the published vocabulary manifest; "other" pools unsupported sentence types and rejected outputs. Six supported captions are correct, and eight background clips are rejected; two background clips are falsely captioned. Background types also occur in training, and a prior baseline outcome on this signer was observed.

13. Limitations, ethics and accessibility

Deployment evaluation requires Deaf-user-led task design, resolved data permissions and consent, independent signers and sessions, novel unsupported utterances, continuous-signing error analysis and measured end-to-end latency. These remain open research and validation requirements.

Small test supports make class metrics unstable. Neither static-image dataset permits a verified signer-disjoint partition in the layout used here, and capture-condition dependence can inflate image-level results. Historical annotations and their display mapping require expert review. The static classifier has no explicit unknown-sign, no-hand or out-of-distribution rejection class; the separate temporal background class does not solve arbitrary unknown recognition. A high softmax score can still be wrong.

KU-BdSL version 4 provides a documented CC BY 4.0 licence and participant-consent statement. That documentation does not transfer to the historical thesis images, whose consent records are absent. The software's MIT licence does not relicense third-party datasets. No unli-

censed face video is incorporated. Historical paper text and figures remain under their original licence, and unchanged v1 is supplied for comparison.

An accessibility claim would require evaluation with Bangla Sign Language users, including variation in signing practice, mobility, lighting, skin tone and camera access. This project has not performed such a study. Static alphabet classification is a constrained research exercise and should not be presented as a replacement for interpreters or a complete communication service.

14. Code and data availability

Code, pinned requirements, manifests, predictions and tests are available at `https://github.com/iamnajib71/Real-Time-Bangla-Sign-Language-Detector`. Versioned models, matching label maps and checksums are provided with release `v2.0.1`: `https://github.com/iamnajib71/Real-Time-Bangla-Sign-Language-Detector/releases/tag/v2.0.1`. The public-data downloader retrieves KU-BdSL v4 directly from its publisher and verifies each supplied SHA256. KU-BdSL v4 is available from its publisher [3] under CC BY 4.0. The historical image records and split manifests are preserved in the repository, although original consent documentation is unavailable. Ban-Sign-Sent source clips remain with the original provider; their reuse permissions are unresolved, and those clips are not redistributed.

15. Changes from version 1

The original preprint is unchanged. This version specifies the executable CNN and its differences from the earlier method description; adds audited partitions, model-release links, per-class metrics and error analysis; and reports separate public-data and bounded sentence-recognition experiments. Original accuracy statements are not treated as results of these experiments. Sparse-autoencoder, depth and MATLAB procedures described previously are not reproduced because corresponding executable evidence is unavailable.

References

[1] N. Hassan. Bangla Sign Language Gesture Recognition System: Using CNN Model. ScienceOpen Preprints, 2022. Non-peer-reviewed. DOI: 10.14293/S2199-1006.1.SOR-.PPUF56Q.v1.

[2] A. A. J. Jim, I. Rafi, M. Z. Akon, U. Biswas and A.-A. Nahid. KU-BdSL: An open dataset for Bengali sign language recognition. Data in Brief 51, 109797, 2023. DOI: 10.1016/j.dib.2023.109797.

[3] A. A. J. Jim et al. KU-BdSL: Khulna University Bengali Sign Language dataset. Mendeley Data, version 4, 2023. DOI: 10.17632/scpvm2nbkm.4.

[4] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen. MobileNetV2: Inverted Residuals and Linear Bottlenecks. 2018. DOI: 10.48550/arXiv.1801.04381.